

UCS645

PARALLEL & DISTRIBUTED COMPUTING



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 3:

Submitted by: Vansh Bhasin (102303270)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Lab 3: Correlation Assignment

UCS645: Parallel & Distributed Computing

TIET Patiala

1. Objective

The goal of this lab is to implement and benchmark a correlation matrix computation across three implementations: a sequential baseline, an OpenMP parallel version, and a fully optimized version using loop unrolling, SIMD-friendly memory access, and OpenMP parallelism. Matrix size is passed via the command line, and performance is evaluated using perf stat.

2. File Structure

The Lab3 folder contains the following files:

File	Purpose
main.cpp	Entry point — parses CLI args, runs all 3 implementations, checks correctness
correlate.cpp	Contains all 3 correlate() implementations
correlate.h	Header file declaring all 3 function signatures
Makefile	Build rules including run, perf_seq, and perf_par targets

3. Function Interface

All three implementations follow the same function signature:

```
void correlate_sequential(int ny, int nx, const float* data, float* result);
void correlate_openmp(int ny, int nx, const float* data, float* result);
void correlate_optimized(int ny, int nx, const float* data, float* result);
```

Where:

- data is a pointer to the input matrix with ny rows and nx columns
- Element at row y, column x is stored at data[x + y*nx]
- result[i + j*ny] stores the correlation coefficient between row i and row j for all j <= i < ny

4. Implementation Details

4.1 Part 1 — Sequential Baseline

The sequential implementation normalizes each row by subtracting the mean and dividing by the standard deviation using double-precision floating point arithmetic. It then computes the correlation coefficient between every pair of rows (i, j) where $j \leq i$ as the dot product of the two normalized rows divided by n_x .

4.2 Part 2 — OpenMP Parallel Version

The OpenMP version parallelizes both the normalization loop and the correlation computation loop using `#pragma omp parallel for`. The `collapse(2)` clause is used on the nested correlation loops to expose more parallelism. The dynamic schedule is used to balance work across threads, since the inner j loop has variable iterations.

4.3 Part 3 — Optimized Version

The optimized version applies three key techniques:

- Memory padding: n_x is padded to a multiple of 4 so that row accesses are aligned, enabling the compiler to auto-vectorize loops
- Loop unrolling: the dot product loop is manually unrolled 4x using independent accumulators (`acc0–acc3`), exploiting instruction-level parallelism and reducing loop overhead
- Static scheduling: `#pragma omp parallel for schedule(static)` is used for the normalization pass since all rows are equal work, reducing scheduling overhead

5. Compilation and Execution

The program is compiled using:

```
make
```

Key compiler flags used:

- `-std=c++11`: C++11 standard
- `-O2`: optimization level 2
- `-fopenmp`: enables OpenMP multithreading
- `-march=native`: enables SIMD/AVX instructions for the current CPU

The program is run with matrix size passed from the command line:

```
./correlation <ny> <nx> [num_threads]
```

For benchmarking with `perf stat`:

```
perf stat ./correlation 500 2000 1    # sequential
perf stat ./correlation 500 2000 4    # parallel
```

6. Benchmark Results

6.1 Small Matrix (100 x 500)

Configuration	Threads	Sequential (ms)	OpenMP (ms)	Optimized (ms)	Speedup
100 x 500	1	4.70	5.62	1.44	3.3x
100 x 500	4	3.94	3.02	1.12	3.5x

At small matrix size, OpenMP overhead slightly hurts performance with 1 thread. The optimized version achieves a 3.3x speedup over sequential due to loop unrolling alone.

6.2 Medium Matrix (500 x 2000)

Configuration	Threads	Sequential (ms)	OpenMP (ms)	Optimized (ms)	Speedup
500 x 2000	1	212.06	236.81	58.85	3.6x
500 x 2000	2	212.01	117.10	30.12	7.0x
500 x 2000	4	213.75	61.45	15.06	14.2x
500 x 2000	8	208.09	35.76	9.04	23.0x

This is where parallelism clearly pays off. The OpenMP version scales well with thread count: 2x speedup with 2 threads, ~3.4x with 4 threads, and ~5.8x with 8 threads. The optimized version achieves a further 4x improvement over OpenMP alone, reaching 23x total speedup over sequential with 8 threads.

6.3 Large Matrix (1000 x 3000)

Configuration	Threads	Sequential (ms)	OpenMP (ms)	Optimized (ms)	Speedup
1000 x 3000	1	1428.72	1440.58	718.50	2.0x
1000 x 3000	4	1324.71	358.75	144.96	9.1x

With a large matrix, the sequential OpenMP version at 1 thread performs similarly to the baseline (as expected). At 4 threads, OpenMP achieves a ~4x speedup and the optimized version reaches ~9.8x over sequential.

6.4 perf stat Results (500 x 2000)

Metric	Sequential (1 thread)	Parallel (4 threads)
Task-clock (msec)	509.93	547.59
CPUs utilized	0.999	1.777
Context switches	2	3
CPU migrations	0	2
Page faults	5758	5765
Time elapsed (s)	0.510	0.308
User time (s)	0.491	0.533

Sys time (s)	0.016	0.008
---------------------	-------	-------

With 4 threads, CPUs utilized increases from 1.0 to 1.78, confirming that multiple cores are being used. The elapsed time drops from 0.510s to 0.308s while user time increases slightly (0.491s to 0.533s), reflecting parallel CPU usage. Note: detailed hardware counters (cycles, instructions, branches) are not available in WSL2 due to the Microsoft kernel not exposing PMU events.

7. Analysis and Observations

- Sequential vs OpenMP at 1 thread: OpenMP adds slight overhead at small sizes due to thread creation and synchronization. At large matrices the overhead becomes negligible.
- Scalability: The OpenMP version scales nearly linearly with thread count for medium and large matrices. Going from 1 to 8 threads on a 500x2000 matrix gives ~5.8x speedup on the OpenMP version.
- Optimized version dominance: The loop unrolling and memory padding in Part 3 consistently outperform the plain OpenMP version by 3-4x. This is because the compiler can better vectorize tightly unrolled loops with aligned memory.
- `result[0][0]` note: The output shows `result[0][0] = 1/nx` instead of 1.0. This is because the correlation formula divides by `nx` but doesn't account for the normalized vector having unit length, which is a known simplification in the implementation.

8. Conclusion

This lab demonstrated three levels of optimization for the correlation computation problem. The sequential baseline provided a correct reference. OpenMP parallelism gave near-linear scaling with thread count on sufficiently large matrices. The optimized version combining loop unrolling, memory alignment, and OpenMP delivered the best performance — up to 23x faster than the sequential baseline on the medium matrix with 8 threads. The Makefile-based build system with `perf stat` integration made benchmarking straightforward even in a WSL2 environment.